Parallel Mining of Frequent Itemsets Based on AprioriHybrid with a Map-Based Trie

Owen X. Kemp and Soon M. Chung
Department of Computer Science and Engineering
Wright State University
Dayton, Ohio 45435, USA

{kemp.56, soon.chung}@wright.edu

***Abstract*—We propose parallelizing the AprioriHybrid algorithm with its Apriori part using a map-based trie to store candidate itemsets. This parallelization is done by distributing the occurrence counting of candidate itemsets across the processes running on different processing nodes. The trie data structure retrieves itemsets faster than the hash tree used in the original Apriori paper. A method was used to transform a trie into and from an array to facilitate sharing itemsets across processes. Our experimental results show a significant speedup with the trie over the hash tree. The parallelized algorithm also showed a significant speedup compared to the sequential algorithm.**

***Keywords*—frequent itemset mining, AprioriHybrid, parallel data mining, trie, performance analysis.**

I. Introduction

Retail organizations collect and store massive datasets of sales data that include transactions of items that consumers bought. These datasets can be used to help analyze trends in consumer habits in order to market better. Finding items that are bought together is the goal of frequent itemset mining. Using these frequent itemsets, association rules can be created to determine the likelihood of a consumer buying a set of items based on what they previously have bought [1]. These rules are very valuable for marketing applications.

Frequent itemset mining was first introduced in [1]. The formal statement of the problem is: Let $I = \{i_1, i_2, \ldots, i_n\}$ be a set of items. Let D be a database be a list of transactions ($D = <t_1, t_2, \ldots, t_m>$). Each transaction t is an ordered set of items made up of items in I ($t \subseteq I$). An ordered set of k items is called a k-itemset, and we assume that items are in ascending order in each itemset and transaction. The support of an itemset X in database D is the number of transactions that contain the itemset, i.e. $support_D(X) = |\{X \subseteq t | \forall t \in D\}|$. An itemset is frequent when its support is greater than a threshold for *minimum support*. The goal of frequent itemset mining is to find all itemsets that are frequent [1].

[2] introduced the Apriori algorithm, an important contribution to the subject. It also introduced variants like AprioriTID and AprioriHybrid. [3] introduced the parallelization of Apriori called Count Distribution, which divides the dataset and synchronizes the processes to find frequent itemsets. [4] introduced using a prefix tree (or trie) data structure to count candidates instead of a hash tree. In this paper, we propose parallelizing AprioriHybrid using a map-based trie data structure to store candidates.

The rest of the paper is organized as follows. Section I described the problem. Section II describes the background information about AprioriHybrid. Section III describes the trie used for Apriori. Section IV describes how parallelization is done. Section V details some implementation details of the data structures used. Section VI describes the performance of the proposed algorithm. Section VII details related algorithms. Section VIII concludes the paper.

II. Background

Conceptually, Apriori generates candidate itemsets and finds their support to determine frequent itemsets. It takes a level-wise approach, starting with finding frequent 1-itemsets and increasing the size of the frequent itemsets after each counting pass over the transaction database [2].

Apriori first determines the frequent items by scanning the database and counting the occurrence of each item. The items that are frequent are in a set L_1. Each subsequent pass k has three phases. It generates the set of candidate k-itemsets, denoted by C_k, based on the set frequent $(k-1)$-itemsets, denoted by L_{k-1}. Next, it will scan the database to determine the support of the candidate k-itemsets. Finally, it will remove any infrequent k-itemsets to obtain L_k containing all the frequent k-itemsets. When no frequent itemsets are found, the algorithm terminates [2].

To generate C_k, L_{k-1} is naturally joined with itself by pairing up all the frequent $(k-1)$-itemsets that share the same first $k-2$ items, except the last item, and adding the two different last items to generate a candidate k-itemset. In the next step, called the prune step, we remove every candidate k-itemset if any of its $(k-1)$-subsets is not in L_{k-1} [2]. For example, if L_2 includes {1, 2} and {1, 3}, these 2-itemsets can be naturally joined into a candidate 3-itemset {1, 2, 3}, However, if {2, 3} is not in L_2, {1, 2, 3} will be pruned.

In [2], a hash tree data structure was used to store candidate itemsets and their support. The root node and internal nodes contain an array of nullable pointers to other internal nodes or

979-8-3315-7578-6/25/$31.00© 2025 IEEE

leaf nodes. The leaf node contains a list of candidate itemsets and their support counts. Traversing the hash tree to find an candidate itemset works as follows: At the root, hash the first item candidate itemset to get an index in the root's array. If the entry points to an internal node, hash the next item in the candidate itemset at the internal node. If the node is a leaf node, then search the candidate itemset in the list. If it exists, then increment its support count; otherwise, add the new candidate itemset in the list

To determine the support of each candidate itemset in a hash tree, each transaction in the database is scanned. In pass k, every k-itemset contained in the transaction would be hashed to the hash tree storing the candidate k-itemsets (and their support counts).

AprioriTID is a modified version of Apriori that performs well on later passes. Conceptually, AprioriTID transforms the database by having each transaction containing numerical identifiers representing candidate itemsets that are contained in the transaction. The transformed database generated at pass k is denoted as $\overline{C_k}$, and conceptually it is in the form of <TID, {ID}>, where TID is the unique transaction identifier, and ID is the identifier of a candidate k-itemset appearing in the transaction. AprioriTID stores an array containing information about each candidate itemset. For each candidate itemset, we store the two identifiers of the two frequent itemsets that were joined to create the candidate itemset, and they are called *generators*. We also store the identifiers of the candidate itemsets generated from an itemset, which are called *extensions* of the itemset [2]. For example, if the identifier of {2, 3} is A, and the identifier of {2, 4} is B, then the generators of {2, 3, 4} are A and B, and the identifier of {2, 3, 4}, say C, would be an extension of {2, 3} and {2, 4}.

AprioriTID first starts by transforming the database by replacing each transaction's items into identifiers, each of which represents one item. This transformed database is $\overline{C_1}$. AprioriTID finds L_1, the set of frequent items (or identifiers in this case) the same as Apriori [2].

For pass k, it generates C_k through self-joining L_{k-1} identifiers and pruning (similar to Apriori). It creates new identifiers with its generators along with the extensions of L_{k-1}'s identifiers. To determine the support of C_k, we use $\overline{C_{k-1}}$ from the previous pass. For each identifier in a transaction, we scan each extension to check whether both generators of that extension are present in the transaction. If so, then the support of that extension is incremented. When scanning each transaction, we create a new transaction containing the identifiers whose support is incremented, and add it to a new modified database $\overline{C_k}$. If no identifier's support has been incremented, then we do not add the transaction to the new database. After filtering frequent identifiers, we get L_k. We continue to the next pass until L_k is empty [2].

AprioriHybrid starts finding frequent itemsets with Apriori. Based on some heuristic, we switch to using AprioriTID at some pass. We can use two different heuristics: The first heuristic checks whether $\overline{C_k}$ can fit into the main memory and that C_k is smaller than C_{k-1}. In this case, we don't need additional disk I/O to produce a modified database. We can estimate the size of $\overline{C_k}$ by summing the support of all C_k itemsets. The second heuristic checks whether the size of $\overline{C_k}$ becomes smaller than the size of original database, so that database scanning time would be reduced. In this paper, we used the first heuristic because it has an advantage when the main memory size is large. If the heuristic passes, AprioriHybrid will switch to AprioriTID. We then recount C_k, in order to generate $\overline{C_k}$ to be used in the next passes. The rest of the algorithm follows AprioriTID [2].

III. Trie

The map-based trie data structure will be used, instead of a hash tree, to store and count the candidate itemsets in Apriori. Tries were originally designed for storing and retrieving text-based information [5]. A trie is a directed tree with the root node being at depth 0. Nodes at depth j contain links to nodes at depth $(j + 1)$, and each link represents an item in an itemset. A map-based trie uses a map to store the links instead of an array. The map's keys are items, and its values are nodes at depth $(j + 1)$. A leaf node is a node without any links to other nodes. A k-itemset is represented as a path from the root to a leaf node at depth $(k-1)$.

To find the leaf node of an itemset, start at the root and traverse the links in the order of the itemset. Inserting an itemset does a similar process, but we create links and nodes when none exists for the itemset. For example, indexing $\{1, 2, 3\}$ into a trie that contains $\{\{1, 2, 3\}, \{1, 2, 4\}, \{2, 3, 4\}\}$ starts at the root node. Then it traverses the links 1, 2, and 3 to get the leaf node corresponding to $\{1, 2, 3\}$.

To determine the support of all the candidate itemsets using a trie, we scan each transaction in the database. A recursive method is used: starting at the root, for each item in the transaction, we find the child node corresponds to that item. If it exits, then call the function recursively on that child node until the node is a leaf. Finally, we increment the count in that leaf node [6].

Once the support for candidate k-itemsets are determined, we can remove the leaf nodes whose k-itemsets are infrequent. This results in a trie containing all the frequent k-itemsets, which can be used to generate the next set of candidate $(k + 1)$-itemsets.

The trie is faster than a hash tree because the comparisons in the leaf node are faster. The hash tree has to iterate through the itemsets in the leaf node of average order $O(n)$, while the trie only has to do a hash map index of average order $O(1)$. The biggest determinant in the speed of the trie is how efficient the hash map is.

IV. Parallelization

In [3], the Count Distribution algorithm is introduced to parallelize Apriori. It finds frequent itemsets by dividing the database into partitions, and each process has one partition to

scan. It first generates C_1 and each process counts the items appearing in its partition. For every pass k, the processes exchange their support counts for C_{k-1}, which is called the synchronization step. Afterwards, all the processes generate L_{k-1} and C_k at the same time, and then count the occurrences of C_k members in their partitions in parallel. The generation of candidate itemsets and finding their support is similar to Apriori. To parallelize AprioriHybrid, we use the AprioriHybrid's methods of candidate generation and support counting instead of the Apriori's methods.

We modified the Count Distribution algorithm by using a special designated process called the coordinator process that performs the synchronization between the processes. This coordinator process receives the local support counts, combines them, and creates the frequent itemsets to send to all the processes. This approach gives the advantage of reducing the network bandwidth usage among the processes.

The coordinator process first sends the rest of the processes their database partitions. It also finds all the frequent items by collecting local support counts of items from the rest of the processes. For each subsequent pass k, the coordinator process broadcasts the frequent $(k-1)$-itemsets to all the processs. The processes generate the candidate k-itemsets and find their local support counts. They send it back to the coordinator process. The coordinator process combines all the local support counts and filters for the frequent itemsets. If no frequent itemsets were found, it sends a termination message to the other processes. Otherwise, the next pass starts.

To exchange the support counts, we converted a trie down into an array. The resulting array's first element is the length of the itemsets being stored. After that, we used a recursive function that takes in a node. First, it appends the number of children to the resulting array. Then, it loops through the node's children, appends the child's item, and recursively calls the method on that child. When it reaches a leaf node, it appends the support count of the leaf (i.e., candidate itemset) to the resulting array.

For example, given a trie containing $\{\{1, 2, 5\}, \{1, 5, 6\}\}$ with corresponding support $\{3, 4\}$, the resulting array could be $\{3, 1, 1, 2, 2, 1, 5, 3, 5, 1, 6, 4\}$. The first and second elements are the length of the candidate itemsets and the number of children in the root node, respectively. The rest of the elements come in pairs of (item, children length) or (item, support count).

The resulting array is then transmitted to the processes to convert back into a trie. To convert the array back into a trie, we use another recursive function. This function takes an iterator over the array, the current node, and depth. We first create the root of the trie and call the recursive function on it with depth being the first element in the iterator. If the depth is zero, then we set the node's value to the next element in the iterator and return. Otherwise, the function gets the next value from the iterator as the length l. It loops l times. Each iteration gets the next element in the iterator as an item. We create the child node corresponding to that element and recursively call the function on it with $depth - 1$.

To exchange frequent itemsets, a similar process is followed with some modifications. The trie used for frequent itemsets is similar to the one used for candidate itemset storage, but the leaf node's value is not used. When converting the trie down to an array, the recursive function does nothing when it reaches a leaf node. When converting the array back into a trie, the recursive function returns when the depth is zero, appending nothing to the resulting array. For the same previous example trie, the resulting array would be $\{3, 1, 1, 2, 2, 1, 5, 5, 1, 6\}$. The only difference is the removal of support counts.

For the parallelization of the support counting in AprioriHybrid, we simply create a trie by inserting the support counts of the identifiers' candidate itemsets into that trie. When it receives the frequent itemsets in a trie, it sets the support of the frequent itemsets' identifiers to maximum values. This allows us to reuse the candidate generation function in AprioriTID. This method also allows processes to switch to using AprioriTID on different passes.

V. Implementation

We implemented all the sequential and parallel algorithms in the Rust language. We made extensive use of Rust's standard library but used an external hash map library *ahash* because the standard library uses a slow hash function. Open MPI [16] is used in the parallel algorithms through the *rsmpi* library.

The dataset is stored as a vector of transactions. Each transaction is stored as a sorted vector of integer items. The trie nodes contain a hash map with items as their key and other trie nodes for their values. Each trie node also contains a field *value*, which can either represent the support count or the presence of a frequent itemset.

For pass one and two of Apriori, we use an array to store the candidate 1-itemsets and candidate 2-itemsets. For pass two, we create a reverse map of the frequent items with the key being the item and the value being the index into the frequent items. This is used to optimize the memory storage of candidate 2-itemsets. The candidate 2-itemsets array is a flattened strictly upper triangular matrix with the array of length $\frac{n*(n-1)}{2}$, where n is the number of frequent items. For later Apriori passes, we use a map-based trie over a hash tree because we found the trie to be faster (See Fig. *1* and Table 1).

For AprioriTID, the transformed database is stored as a vector of transaction identifiers. Each transaction identifier is stored as a hash set of integer identifiers. The identifier information is stored in an array indexed by the identifier.

A small optimization can be made in AprioriTID by only adding one extension when creating new identifiers. This halves the number of iterations the loop goes through when determining the support of C_k. This works because we still check the same identifiers when counting.

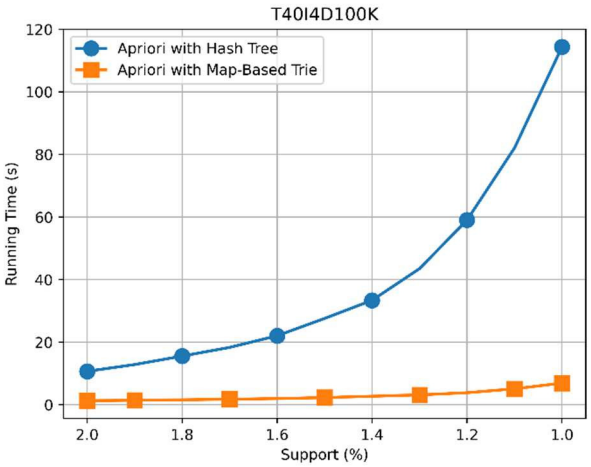

Fig. 1: Comparison of Data Structures on a T40I4D100K Dataset

Table 1: Comparison of Data Structures on a T40I4D100K Dataset

Support (%)	Apriori with Hash Tree (s)	Apriori with Map-Based Trie (s)
2.0	10.69	1.31
1.9	12.85	1.43
1.8	15.56	1.59
1.7	18.33	1.77
1.6	22.05	1.98
1.5	27.59	2.29
1.4	33.38	2.72
1.3	43.51	3.16
1.2	59.04	3.85
1.1	82.08	5.10
1.0	114.39	6.92

VI. Experimental Results

We present the experimental results of our implementation of the parallel AprioriHybrid in comparison with other algorithms discussed here: Apriori, AprioriHybrid, and parallel Apriori.

We ran these algorithms on an HPC server containing Intel Xeon E5-2620 CPUs, and each CPU has 180 GB of RAM. They are connected to each other by a 200 GB/s Omnipath network and a 10 GB/s Ethernet network. One node of the HPC contains a CPU and its memory. The HPC was running the Linux operating system with a Singularity image file containing Ubuntu 24.04. The parallel algorithms were ran with 8 nodes and 4 concurrent processes per node.

We tested the algorithms on two databases: T40I4D1M and *Kosarak*. T40I4D1M contains one million transactions with an average transaction size of 40 items. The total number of items included is 1000. The average size of potentially frequent itemsets is 4 items. *Kosarak* contains 990,002 transactions whose average size is 8.1 items.

Our experimental results show that Apriori and AprioriHybrid performed similarly on larger transaction database sizes like T40I4D1M (See Fig. 2 and Table 2) because the time in earlier passes were dominant. On smaller, sparser transaction databases like *Kosarak*, AprioriHybrid performed significantly better than Apriori (See Fig. 3 and Table 3). Both parallel algorithms performed similarly on both databases. Parallel Apriori had a speedup of around 10 times over Apriori on the T40I4D1M database. We also tested the parallel algorithms with more concurrent processes per node, but there was no additional speedup due to the increased memory contention at each node.

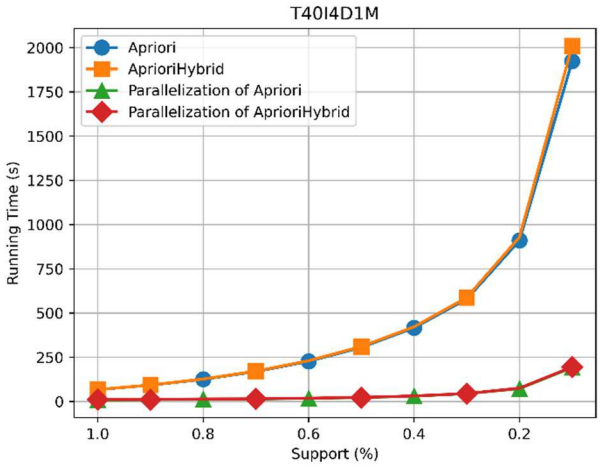

Fig. 2: Performance on a T40I4D1M Dataset

Table 2: Performance on a T40I4D1M Dataset

Support (%)	Apriori (s)	AprioriHybrid (s)	Parallelization of Apriori (s)	Parallelization of AprioriHybrid (s)
1	67.15	67.90	9.47	11.41
0.9	92.62	93.55	11.60	11.10
0.8	125.53	127.23	13.22	14.49
0.7	169.38	171.96	14.59	15.26
0.6	227.15	230.65	18.02	19.01
0.5	306.00	310.98	22.46	22.95
0.4	416.42	422.25	30.50	31.62
0.3	579.29	587.82	44.37	45.03
0.2	909.89	928.73	73.07	75.66
0.1	1923.03	2008.91	192.26	195.38

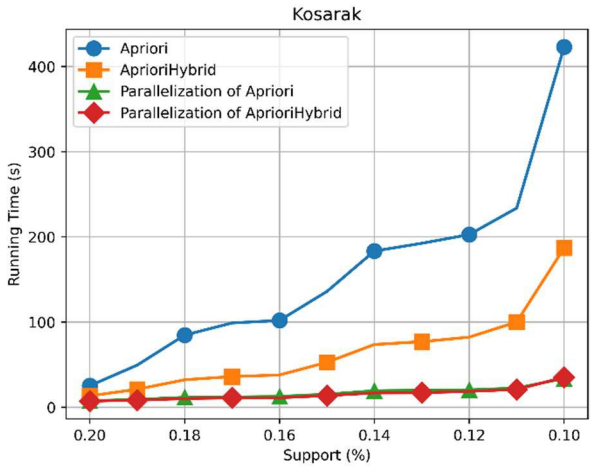

Fig. 3: Performance on the Kosarak Dataset

Table 3: Performance on the Kosarak Dataset

Support (%)	Apriori (s)	AprioriHybrid (s)	Parallelization of Apriori (s)	Parallelization of AprioriHybrid (s)
0.2	25.30	13.39	7.52	7.27
0.19	49.51	21.14	9.30	8.39
0.18	84.73	32.29	11.67	10.19
0.17	99.00	36.10	12.07	11.19
0.16	101.94	37.91	12.75	11.16
0.15	136.00	52.83	15.37	13.77
0.14	183.44	73.72	19.41	16.87
0.13	192.52	77.02	20.09	17.09
0.12	202.75	82.38	20.50	18.71
0.11	233.88	100.17	22.55	20.98
0.1	423.04	187.10	33.87	35.28

The scalability was tested on the T40I10D1M database, and the experimental results are shown in Fig. 4 and Table 4. We tested it using the AprioriHybrid and its parallelization where each node had 4 processes.

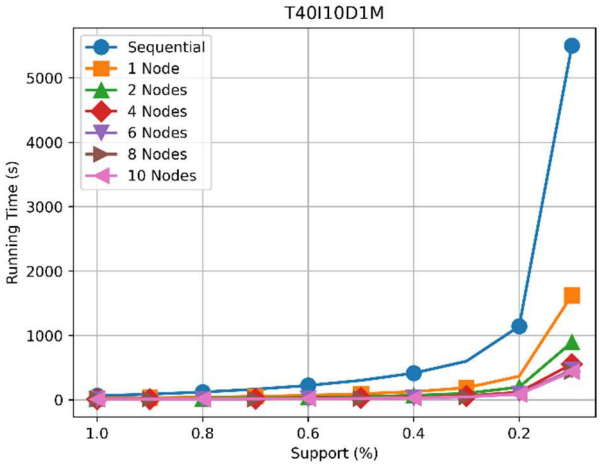

Fig. 4: Performance of Various Number of Nodes on a T40I10D1M Dataset

Table 4: Performance of Various Number of Nodes on a T40I10D1M Dataset

Support (%)	Sequential	1 Node (s)	2 Nodes (s)	4 Nodes (s)	6 Nodes (s)	8 Nodes (s)	10 Nodes (s)
1.0	64	26	16	12	11	18	10
0.9	89	33	20	14	12	12	12
0.8	122	43	24	16	13	13	12
0.7	166	54	31	20	17	15	15
0.6	224	71	39	25	20	18	19
0.5	303	95	52	31	26	23	23
0.4	416	128	69	42	34	31	30
0.3	600	189	102	62	50	47	45
0.2	1143	367	198	119	96	89	88
0.1	5502	1626	896	556	466	442	440

Our analysis of the running time shows that as the number of nodes increases, the time for the coordinator process to combine the local support counts of candidate itemsets takes longer. This causes a bottleneck in the consolidation step where all the other processes have to wait. See Fig. 5 and Fig. 6 for the difference.

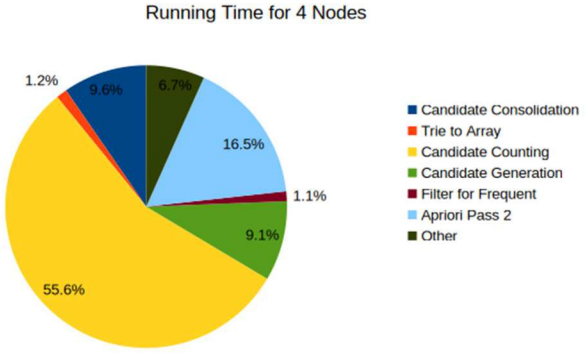

Figure 5: Breakdown of the Running Time with 4 Nodes on T40I10D1M with Minimum Support 0.1%

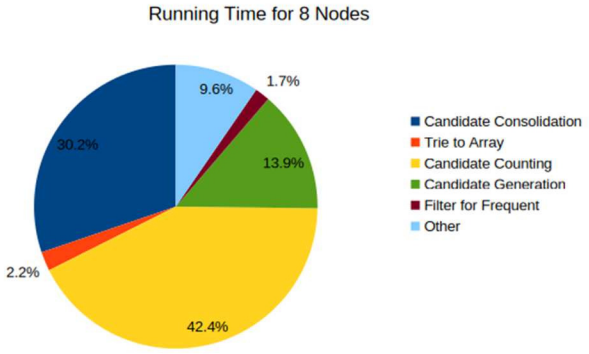

Figure 6: Breakdown of the Running Time with 8 Nodes on T40I10D1M with Minimum Support 0.1%

Our results show that the parallel AprioriHybrid algorithm has a significant speedup over the sequential version, but it does not have good scalability due to the increasing cost of the consolidation of candidate itemsets.

VII. Related Work

Many other frequent itemset mining algorithms have been proposed. [7] proposed the FP-Growth which uses a FP-tree data structure to mine frequent itemsets. This eliminates the need to scan through the database each pass. [8] introduced the LCM algorithm, which uses several optimizations to minimize redundant searches. [9] proposed the ECLAT algorithm, which uses a vertical data format to reduce database scans. [10] parallelizes ECLAT on Spark RDD framework.

Many more modern algorithms focus on increasing scalability on parallel processing systems. [11] introduces the BigMiner algorithm that uses transaction chunks to achieve higher scalability. [12] introduces PFIMD that parallelizes DiffNodeset [13] using the MapReduce model. [14] parallelizes PrePost$^+$ [15] using FP-tree-like data structures.

VIII. Conclusion

Finding frequent itemsets is an important task for data mining. The data structure used to store and count candidate itemsets impacts the performance of the Apriori algorithm. In this paper, we used a map-based trie data structure. We found

that the trie is more efficient than the hash tree originally proposed in [2]. We also implemented AprioriHybrid to see whether it is faster than Apriori with a map-based trie. We found that it is faster only on certain types of databases.

We also showed parallel versions of Apriori and AprioriHybrid, both with a map-based trie. These versions are modified versions of the Count Distribution algorithm introduced in [3]. These parallel versions exchange support counts and frequent itemsets through transforming a trie into and from an array. They also use a coordinator process that synchronize the other processes. The parallel algorithms showed significant speedups over their sequential versions. We found that the parallel algorithms have similar performances on certain databases and minimum support levels.

As different databases have different characteristics, no single mining algorithm can be optimal for all databases. Understanding the properties of the database is crucial in determining what algorithms would perform best on it. Future work could be on the analysis of database properties that affect the performance of mining algorithms and also on parallelizing the consolidation of the candidate itemsets into a single trie, in order to increase scalability. Applying a combination of MPI and OpenMP (Open Multi-Processing) [17], such that using MPI between processors, whereas using OpenMP between threads at each processor, could be another interesting topic.

REFERENCES

[1] R. Agrawal, T. Imieliński, and A. Swami, "Mining Association Rules between Sets of Items in Large Databases," ACM SIGMOD Record, vol. 22, no. 2, pp. 207–216, June 1993. doi: 10.1145/170036.170072.

[2] R. Agrawal and R. Srikant, "Fast Algorithms for Mining Association Rules in Large Databases," Proc. of the 20th Int'l Conf. on Very Large Data Bases (VLDB 1994), pp. 487–499, 1994.

[3] R. Agrawal and J. C. Shafer, "Parallel Mining of Association Rules," IEEE Trans. on Knowledge and Data Engineering, vol. 8, no. 6, pp. 962–969, Dec. 1996. doi: 10.1109/69.553164.

[4] F. Bodon and L. Rónyai, "Trie: An Alternative Data Structure for Data Mining Algorithms," Mathematical and Computer Modelling, vol. 38, no. 7–9, pp. 739–751, Oct. 2003. doi: 10.1016/0895-7177(03)90058-6.

[5] E. Fredkin, "Trie Memory," Communications of the. ACM, vol. 3, no. 9, pp. 490–499, Sept. 1960. doi: 10.1145/367390.367400.

[6] F. Bodon, "A Fast Apriori Implementation.," Proc of Workshop on Frequent Itemset Mining Implementations, 2003.

[7] J. Han, J. Pei, and Y. Yin, "Mining Frequent Patterns without Candidate Generation," *ACM SIGMOD Reord*, vol. 29, no. 2, pp. 1–12, 2000. doi: 10.1145/335191.335372.

[8] T. Uno, M. Kiyomi, and H. Arimura, "LCM ver. 2: Efficient Mining Algorithms for Frequent/Closed/Maximal Itemsets," Proc of Workshop on Frequent Itemset Mining Implementations, 2004.

[9] M. J. Zaki, "Scalable Algorithms for Association Mining," IEEE Trans. on Knowledge and Data Engineering, vol. 12, no. 3, pp. 372–390, 2000.

[10] P. Singh, S. Singh, P. Mishra, and R. Garg, "RDD-Eclat: Approaches to Parallelize Eclat Algorithm on Spark RDD Framework," Proc. of Int'l Conf. on Computer Networks and Inventive Communication Technologies, Springer, 2019, pp. 755–768.

[11] K.-W. Chon and M.-S. Kim, "BIGMiner: A Fast and Scalable Distributed Frequent Pattern Miner for Big Data," Cluster Computing, vol. 21, no. 3, pp. 1507–1520, 2018.

[12] M. Yimin *et al.*, "PFIMD: A Parallel MapReduce-based Algorithm for Frequent Itemset Mining," Multimedia. Systems, vol. 27, no. 4, pp. 709–722, 2021. doi: 10.1007/s00530-020-00725-x.

[13] Z.-H. Deng, "DiffNodesets: An Efficient Structure for Fast Mining Frequent Itemsets," Applied Soft Computing, vol. 41, pp. 214–223, 2016.

[14] V. Q. P. Huynh and J. Küng, "FPO Tree and DP3 Algorithm for Distributed Parallel Frequent Itemsets Mining," Expert Systems with Applications, vol. 140, 2020. doi: 10.1016/j.eswa.2019.112874.

[15] Z.-H. Deng and S.-L. Lv, "PrePost+: An Efficient N-lists-based Algorithm for Mining Frequent Itemsets via Children–Parent Equivalence Pruning," Expert Systems with Applications, vol. 42, no. 13, pp. 5424–5432, 2015. doi: 10.1016/j.eswa.2015.03.004.

[16] E. Gabriel, G. E. Fagg, G. Bosilca et al., "Open MPI: Goals, Concept, and Design of a Next Generation MPI Implementation," Proc. of EuroPVM/MPI, LNCS 3241, 2004, pp. 97–104.

[17] "OpenMP Compilers & Tools," available at: https://www.openmp.org/resources/openmp-compilers-tools/